

SEEING THE PATTERNS, CLOSING THE LOOP: EVOLVING ENVIRONMENTS FOR AGENT TRAINING

Anonymous authors

Paper under double-blind review

ABSTRACT

LLM agents can serve as data researchers, automating the creation of training data for LLM post-training. We study data-centric recursive self-improvement for agent post-training. In this setting, a researcher agent starts from an empty environment library and constructs environments for online RL without access to the target backend or verifier; evaluation of the updated model then supplies feedback for the next round. Our experiments and audits identify three challenges: incomplete behavioral analysis, disconnected data–training–evaluation feedback, and incomplete implementation of intended environment mechanisms. We introduce ENVOPT, which uses source-traceable hierarchical analysis to *see the patterns* in agent behavior and target-conditioned context refinement to *close the loop* between data, training, and evaluation. These two designs guide environment construction and revision, supported by specification-driven implementation and independent review. Preliminary experiments with an earlier implementation yield observed test gains of 16.67 and 11.11 percentage points on BFCL and ACEBench, respectively, although gains vary across rounds. These findings suggest that evidence-guided environment evolution is a promising direction for automated agent post-training.

1 INTRODUCTION

Recursive self-improvement (RSI) asks whether AI systems can use evidence from one development cycle to improve the next. MetaRSI[CITE: [MetaRSI](#)] distinguishes three writable surfaces for improvement: data, the execution harness, and model parameters. We focus on the interaction between the data and model surfaces, where an agent adapts training data as the learner evolves. Relatedly, RSIBench-Data ([Meng et al., 2026](#)) studies agents that revise training-data strategies across training runs and reveals a discovery–reliability gap: agents can discover useful strategies but struggle to improve them consistently. This raises a sharper question: when the learner itself changes, can feedback from one training round reliably guide data construction for the next?

We study this question in the context of training agents to use tools more effectively, as tool use is a fundamental capability for modern agents. Training such agents extend beyond generating static samples because training data take the form of executable instances that define state, tool behavior, tasks, and outcome verification. Systems for executable environment synthesis and scaling, including AgentScaler, EnvScaler, ScaleEnv, Agent World Model, and EnvFactory ([Fang et al., 2025](#); [Song et al., 2026](#); [Tu et al., 2026](#); [Wang et al., 2026](#); [Xu et al., 2026](#)), show that such resources can be constructed automatically. Building on this capability, we study data-centric recursive improvement in a stateful, multi-round online-RL setting, where a target model is iteratively updated using new data produced by a data-researcher agent. Specifically, starting from an empty pool of executable environments, the agent operates in a four-stage loop: analyzing the current target model’s behavior, constructing executable training instances, submitting them to a fixed external service for online-RL training and evaluation, and using the resulting training records and evaluation feedback to guide the next round. With each round continuing from the updated model, the training data and target policy evolve together over time. Reliable recursive improvement is therefore not a one-shot data-generation problem: it requires sustaining a coherent data loop as both continue to evolve. Our experiments show that even a modern agent with strong reasoning and coding capabilities (Codex with GPT-6 Astra) struggles to sustain improvement across rounds. The target model’s performance often plateaus or regresses, and some runs even end in training collapse.

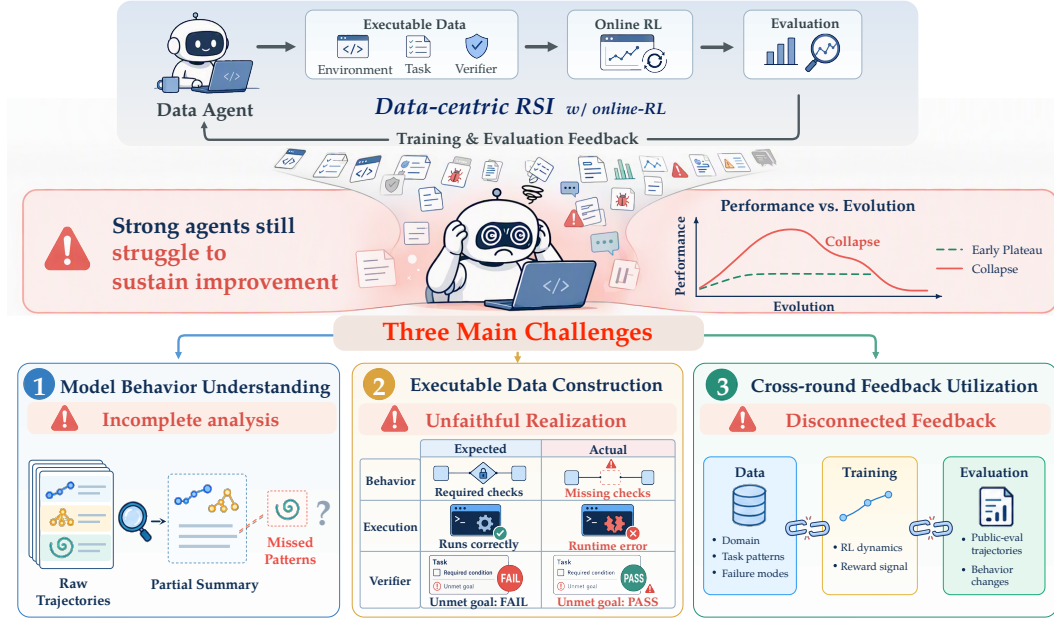


Figure 1: Three challenges in closing the data-centric RSI loop: incomplete behavioral analysis, unfaithful realization of executable training instances, and disconnected data–training–evaluation feedback. Illustrations and records are schematic.

Based on our experiments and observations, we identify three key stages in closing the data improvement loop and their associated failure modes:

- **Model behavior understanding.** *Incomplete analysis* of available traces yields a partial or biased understanding of the target model.
- **Cross-round feedback utilization.** *Disconnected feedback* prevents the agent from relating constructed data to realized training experience and subsequent target behavior.
- **Executable data construction.** *Unfaithful realization* produces environments that are logically inconsistent or misaligned with the targeted domain or capability.

To these ends, we introduce ENVOPT, a framework that addresses these failure modes through three corresponding components. To see patterns in target model behavior, a **traceable behavioral map** consolidates evidence across trajectories while preserving links to the source cases. To close the feedback loop across rounds, **target-conditioned context refinement** organizes relevant data, observed training dynamics, and subsequent evaluation outcomes around each training target. To faithfully realize each training target as executable data, **iterative environment construction** uses a planning, implementation, and refinement loop to bridge the last mile from target specification to logically consistent and behaviorally aligned environments. Extensive experiments across four agentic tool-use benchmarks demonstrate the effectiveness of ENVOPT for multi-round improvement. On Qwen3-4B, ENVOPT achieves substantial performance gains of +7.44, +16.67, +11.11, and +13.29 percentage points on τ^2 -Bench, BFCL-v4, ACEBench-Agent, and AppWorld, respectively. Across successive iterations, target model performance with ENVOPT follows a clear overall upward trend, whereas the same Codex researcher without these mechanisms produces irregular fluctuations with no sustained direction of improvement. Further analyses show that the three proposed designs improve their corresponding stages of the loop by increasing coverage of task requirements and failure mechanisms, improving the quality of realized RL signals, and producing executable environments that are more reliable and better aligned with their training targets.

2 RELATED WORK

Data-centric RSI. RSIBench-Data studies agents that revise training-data strategies for a separate target model under fixed training, evaluation, and resource controls (Meng et al., 2026). Autodata develops an agentic data scientist that iterates between data creation and quality assessment, including meta-optimization of the data scientist itself (Kulikov et al., 2026). DataMaster combines data discovery and transformation with tree search and cumulative memory while holding the learning algorithm fixed (Du et al., 2026). ENVOPT studies this research role for executable training environments: a researcher starts with an empty environment library and uses behavioral evidence to guide successive rounds of environment revision and online RL. Its focus is the evidence supporting those research decisions.

Executable environment synthesis for tool-calling agents. AgentScaler, EnvScaler, and ScaleEnv automate the construction of executable tools, state, and verifiable tasks (Fang et al., 2025; Song et al., 2026; Tu et al., 2026). Agent World Model scales database-backed synthetic environments for RL, while EnvFactory combines executable environment construction with topology-aware task synthesis (Wang et al., 2026; Xu et al., 2026). FACET grounds task instructions, solutions, and verifiers in shared executable state to preserve source intent and cross-artifact consistency (Shi et al., 2026). The ACE framework and WRIT further examine data accuracy, learner-relative complexity, diversity, and the evidence required for individual tool-use decisions (Zeng et al., 2026; Gu et al., 2026). These works establish construction and validation principles that ENVOPT builds on; our emphasis is using source-traceable behavioral analysis and actual-training feedback to decide which mechanisms to construct or revise.

Iterative RL and environment adaptation. PAIRED and Prioritized Level Replay adapt environment generation or selection using regret and learning-potential signals (Dennis et al., 2020; Jiang et al., 2021). For tool-use agents, RODS uses online reward statistics to synthesize structurally aligned tasks and update a replay pool (Fang et al., 2026). EnvHarness adapts existing environments from policy trajectories while retaining their original verifiers (Huang et al., 2026). SPADE trains a shared environment-designer and solver model through self-play, using hint-based regret, corpus grounding, and environment memory (Liu et al., 2026). Agent-World and AgentOmnia also use behavioral diagnosis to guide targeted task synthesis and self-evolution (Dong et al., 2026; Jiang et al., 2026). ENVOPT uses an external researcher to revise independent environments without the target backend or verifier. It connects the source evidence behind a diagnosis to realized program versions, training tasks, RL observations, and fixed-target evaluation, supporting inspection of what each intervention actually delivered.

3 AGENTS AS DATA RESEARCHERS: IDENTIFYING CHALLENGES IN DATA-CENTRIC RSI

We instantiate data-centric RSI as an iterative environment-generation and reinforcement-learning loop. An agent serves as the data researcher, constructing executable environments and tasks in which a target model trains with online RL. Feedback from training and evaluation informs the next round. We first formalize this process, then examine challenges in its research context and generated training instances.

Formalizing the data-research loop. A *research context* is the information available to an agent for research decisions, encompassing the research history, current state, and other decision-relevant information. Let C_t denote this context at round t and DataResearcher the agent that uses it to construct training instances. Each instance is a triple (e, x, v) , consisting of an executable environment e , a task x specifying a goal and initial conditions, and a verifier v assessing task completion. The agent’s output is a set D_t of n_t such instances:

$$D_t = \{(e_j, x_j, v_j)\}_{j=1}^{n_t} = \text{DataResearcher}(C_t). \quad (1)$$

The set D_t is then submitted for training and evaluation. Let π_t denote the target model at the start of round t and TrainEval the fixed online-RL and public-evaluation procedure. It returns the updated model π_{t+1} and feedback F_t comprising training records and evaluation results:

$$(\pi_{t+1}, F_t) = \text{TrainEval}(\pi_t, D_t). \quad (2)$$

Finally, `UpdateContext` integrates the submitted set and its feedback with the prior research context to guide the next round:

$$C_{t+1} = \text{UpdateContext}(C_t, D_t, F_t). \quad (3)$$

In this setting, the loop starts from an empty environment library. Guided by feedback and the evolving research context, the data researcher repeatedly creates new environments or revises existing ones, with the goal of improving the target model over successive training rounds.

A generated environment is an executable program represented by

$$e = (\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{O}),$$

where $\mathcal{S}$ contains its data and state, $\mathcal{A}$ defines the available tools, $\mathcal{T}$ specifies their effects on state, and $\mathcal{O}$ determines the tool outputs and user responses visible to the model. The task x fixes a goal and initial conditions, and the verifier v checks the state and interaction trace to provide the training reward.

3.1 INCOMPLETE BEHAVIORAL ANALYSIS

The research context must preserve task requirements and model behavior to support training-target selection. To examine what survives this analysis, we give an analyst agent (Astra/high) complete access to the initial public-eval trajectories of Qwen3-8B reasoning: 1,200 BFCL, 417 τ^2 , and 75 ACEBench-Agent trajectories. The instruction requests an analysis and specifies report fields, while leaving the reading strategy to the agent. We measure report coverage of source-grounded task requirements and failure mechanisms, crediting full preservation of their consequential conditions and dependencies (Section 5.3).

The analysis reveals a substantial gap: task-requirement coverage is 61.5–82.5%, while failure-mechanism coverage ranges from only 39.4% on BFCL to 67.5% on ACE. These automatic, task-averaged scores use the same final paired evaluation as Section 5.3; the remainder includes both partial descriptions and wholly missing mechanisms. Full-corpus access thus leaves substantial behavioral information incompletely represented. Coverage is lower on the larger corpora in this comparison, although benchmark composition and corpus size vary together, preventing a causal size claim.

The research context therefore needs to preserve the mechanisms and conditions that matter for the next data decision, with evidence available for inspection.

3.2 DISCONNECTED DATA, TRAINING, AND EVALUATION

Updating the research context requires connecting intended training targets, tasks actually trained, and subsequent evaluation outcomes. In ACE R9, one revision contributed zero of its ten planned tasks. We further audited 28 iterations of the original 4B studies, measuring the fraction of trained tasks with nonconstant rewards across eight valid rollouts—the within-task contrast used by GRPO. Among complete, valid groups, the task-weighted rates were 74.6%, 42.2%, and 37.2% on τ^2 , BFCL, and ACE. These checks distinguish whether an intervention reached training and whether its rollouts supplied reward contrast, motivating explicit linkage to its intended target and evaluation outcomes. Section 5.4 compares the per-round signal with Codex-Vanilla.

3.3 INCOMPLETE ENVIRONMENT IMPLEMENTATION

A retail-exchange environment required matching identical items to different replacements, yet all six inspected tasks assigned the same replacement to both items. Checking the generator against the specification confirmed that the required branch was absent. This exposes a gap in constructing the output triples: a runnable instance need not realize the intended training mechanism. Auditing environment behavior, task requirements, and verifier scores tests whether the three components jointly deliver the training target. [TODO: Cross-benchmark audit: 20 environments each on τ^2 , BFCL, and ACE; eight rollouts per environment; LLM judgments of target implementation, interaction, and verifier defects. Insert observed defect rates.]

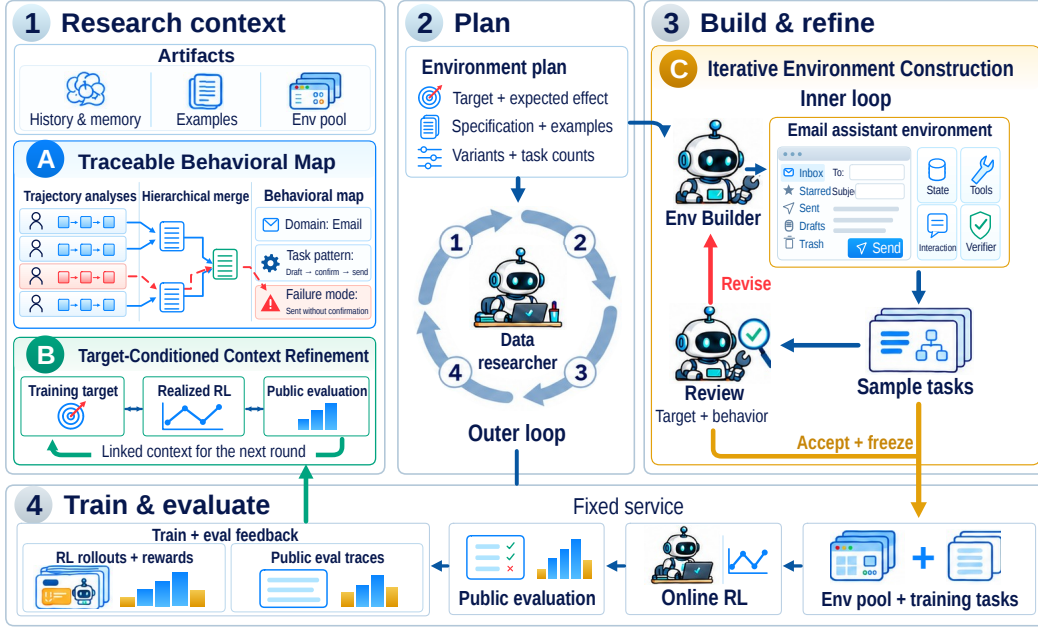


Figure 2: ENVOPT’s research loop. A/B expand research artifacts; C shows iterative environment construction. The outer loop connects research context, planning, construction, and fixed RL/public evaluation. Implementation findings return to the same Env Builder; accepted tasks are frozen before RL. Email-assistant examples, curves, and evaluation marks are schematic.

4 METHOD

ENVOPT instantiates the data-research loop in Equations 1–3 (Figure 2). Its two core designs organize the research context C_t : a *traceable behavioral map* preserves task requirements and failure mechanisms with links to source trajectories; *target-conditioned context refinement* uses target-linked feedback to build decision-ready context for comparing intended training, actual RL experience, and target-evaluation outcomes. Guided by this context, specification-driven construction and independent review produce the environment–task–verifier set D_t . The training-and-evaluation service remains fixed.

The following describes the maintained framework and labels a standalone analysis refinement. Section 5 distinguishes these from the earlier protocols underlying the reported training curves.

4.1 TRACEABLE BEHAVIORAL MAP

The map separates a persistent evidence layer from its compact reading view. Let $T_{t,i}$ contain the complete public-eval episodes for task i , including instructions, tool definitions, and observed interactions. Extraction and source review produce an immutable mechanism ledger:

$$\mathcal{L}_t = \bigcup_i \text{ExtractReview}(T_{t,i}). \quad (4)$$

Each record links an environment mechanism, task requirement, or observed failure/recovery to its source and applicable conditions.

Bounded hierarchical merging builds a draft by factoring shared premises while retaining distinct actors, scopes, dependencies, and recovery branches:

$$R_t^{(0)} = \text{FactorMerge}(\mathcal{L}_t). \quad (5)$$

The draft is checked against the retained evidence, not only earlier summaries. Repair locally corrects conflicting claims; Residual extracts and consolidates only relations still missing from the

corrected report:

$$\begin{aligned}\bar{R}_t &= \text{Repair}(R_t^{(0)}, \mathcal{L}_t), \\ R_t &= \bar{R}_t \oplus \text{Residual}(\mathcal{L}_t, \bar{R}_t).\end{aligned}\tag{6}$$

Here $\oplus$ appends the missing distinctions without rewriting preserved content; an already-covered record must instead be matched to an existing passage. These are model-based checks, not a semantic-completeness guarantee.

The report retains the four fields *domain*, *environment*, *task behavior*, *failure behavior*. It enters the research context C_t with links back to original trajectories. This coverage-checked variant is evaluated offline in Section 5.3, separately from the earlier analysis used in the historical training runs.

4.2 TARGET-CONDITIONED CONTEXT REFINEMENT

Access to evidence does not ensure its use in research decisions. We target this *information-utilization gap* with *decision-ready context*: evidence organized for joint interpretation around a shared behavioral target. Our *target-linked feedback* provides the context-level intervention, explicitly connecting what was intended, what was trained, and how target behavior changed.

For each intervention, the researcher declares a target requirement and expected effect, anchored to public-eval tasks. Versioned triples in D_t and their task IDs link this target to actual RL groups and rollouts, forming a target-conditioned evidence view:

$$\underbrace{\text{Training target}}_{\text{requirement, expected effect}} \longleftrightarrow \underbrace{\text{Actual training}}_{\text{versioned triples, RL rollouts}} \longleftrightarrow \underbrace{\text{Target evaluation}}_{\text{linked public tasks}}.\tag{7}$$

The view distinguishes planned, accepted, and actually trained data, pairing training-time reward statistics and within-task reward contrast with linked public-eval outcomes and trajectories. The researcher compares these observations with the expected effect, recording agreements, mismatches, and unresolved questions in C_{t+1} . This assessment guides revisions to mechanisms, challenge, or sampling, and decisions to retain or pause environments. The design guides evidence use rather than adding otherwise inaccessible feedback; the links do not establish causal effects of individual edits.

4.3 ITERATIVE ENVIRONMENT CONSTRUCTION

We construct training environments through an iterative cycle of planning, implementation, and review, revising either the implementation or the plan as needed. The researcher selects training targets and task allocations, then provides specifications with success, failure, and recovery cases. Persistent Env Builders receive these specifications rather than raw public-eval or RL corpora.

Each builder implements candidate (e, x, v) triples, with task requests and verifiers sharing the same goal data. Execution checks and independent review examine the specification, full program, and all tasks in the batch. They assess whether the intended decisions and variations are implemented and whether information access, tool effects, and verifier rewards are consistent. Implementation findings return to the same builder for revision and renewed review, retaining prior findings; specification ambiguities return to the researcher to revise the plan.

Accepted triples are frozen and retained while the researcher replans any remaining allocations, subject to construction budgets. The accepted set D_t enters training once its admission requirement is met. This local construction cycle precedes the outer training and feedback loop; review remains fallible, not a proof of program correctness.

5 EXPERIMENTS

5.1 EXPERIMENTAL SETUP

Evaluation and training. We evaluate on τ^2 -Bench (Barres et al., 2026), BFCL-v4 (multi-turn) (Patil et al., 2025), ACEBench-Agent (Chen et al., 2025), and AppWorld (Trivedi et al., 2024).

The first three benchmarks use 1:1 public/private splits; AppWorld uses its train split for public evaluation and test-normal split for private evaluation. Public evaluation informs environment evolution and checkpoint selection, while private evaluation is withheld from the evolution process and used for final performance comparisons. Target models train with GRPO (Shao et al., 2024).

Implementation and baselines. Our target models are Qwen3-4B-Instruct-2507 and Qwen3-8B (Yang et al., 2025), with reasoning enabled for the latter. Our data researcher agent, DataResearcher, is instantiated with Codex powered by GPT-6 Astra. We grant the agent access to the GPT-5.5 API and 4 A100 GPUs during evolution. We compare against two baselines: vanilla Codex as an autonomous data researcher (Codex-Vanilla) and training on a fixed environment set generated by DataResearcher without revision (Fixed-Env). Both baseline researcher agents are powered by GPT-6 Astra.

5.2 MAIN RESULTS

Strong coding agents struggle with continual data evolution. Despite pairing GPT-6 Astra with the Codex harness, Codex-Vanilla shows early gains followed by saturation or regression (Figure 3). On BFCL, public performance peaks at 30.00% in R3 and falls to 25.42% by R10. On τ^2 , it finishes at 21.58%, below its 27.10% starting score.

A plausible bottleneck is maintaining the learning value of generated environments as the target model changes. This requires updating training targets from behavioral feedback and faithfully implementing the corresponding mechanisms, the challenges identified in Section 3. Incomplete implementations or poorly updated targets may instead produce executable environments that offer little additional learning signal.

Substantial gains across four benchmarks. ENVOPT improves 4B public-evaluation performance by +23.74, +22.58, +30.00, and +26.75 percentage points over the base model on τ^2 -Bench, BFCL-v4 (multiturn), ACEBench-Agent, and AppWorld, respectively. These results use the earliest public maximum within the displayed rounds. The contrast is clearest on BFCL: ENVOPT reaches 45.00% in R9, well beyond Codex-Vanilla’s 30.00% peak in R3. On both benchmarks with baseline curves, ENVOPT maintains higher scores throughout the shared R3–R9 rounds. All four displayed endpoints remain above their starting scores. Table 1 reports test performance; the 4B Codex-Vanilla entries use R9 on τ^2 -Bench and BFCL-v4, and R10 on AppWorld.

5.3 BEHAVIORAL COVERAGE AND REPORT COMPACTNESS

We now test whether a structured analysis process reduces the information loss identified in Section 3.1. On the same 1,692 frozen initial Qwen3-8B public-eval trajectories, we compare the fixed coding-agent reports (Astra/high) with a standalone, coverage-checked refinement of the behavioral map (GPT-5.5/high). The refinement analyzes complete task evidence, consolidates shared mechanisms, and checks the compact report for omitted conditions. It receives neither the reference labels nor the coding agent’s report prose. We extend the same protocol to 243 initial public-eval trajectories of Qwen3-4B-Instruct-2507 on AppWorld.

Measure retained meaning, not compression alone. Frozen, source-grounded reference units describe task requirements and failure mechanisms. An LLM judge scores anonymized report pairs as full, partial, missing, or contradicted. Only full preservation of the mechanism and its consequential scope or dependency receives credit. We average units within each task and then tasks within each corpus; word count separately measures compactness. References are generated before seeing candidate reports, not inferred from either method’s preferred categories.

Higher coverage in shorter reports. The refined map reaches 88.1–99.5% task-requirement coverage and 71.3–88.4% failure-mechanism coverage, while using 12–29% fewer words (Figure 4a). It exceeds the paired grades of the coding-agent reports on all four corpora; the ordering also holds under subset–family-balanced aggregation. A concrete ACE example shows why this is more than naming more error types: an inbox-cleanout timeout label omits the user-authorization–deletion–retry dependency, which the refined report preserves. Both graders support partial versus full coverage for that example. Not every case improves.

Table 1: Test performance (%) at the reported checkpoints. Green/red deltas show gains/drops from None (pp); Avg. averages all four benchmarks. Dashes indicate unreported results.

Method	τ^2 -Bench	BFCL-v4	ACEBench-Agent	AppWorld	Avg.
<i>Qwen3-4B-Instruct-2507</i>					
None	31.89	25.42	41.11	8.93	26.84
Codex-Vanilla	21.58 -10.31	25.75 +0.33	—	13.69 +4.76	—
Fixed-Env	27.58 -4.32	33.17 +7.75	45.00 +3.89	5.56 -3.37	27.83 +0.99
ENVOPT	50.60 +18.71	42.08 +16.67	52.22 +11.11	22.22 +13.29	41.78 +14.94
<i>Qwen3-8B (reasoning enabled)</i>					
None	28.54	44.42	43.89	17.86	33.68
Codex-Vanilla	—	—	—	—	—
Fixed-Env	—	—	—	—	—
ENVOPT	37.41 +8.87	—	—	24.01 +6.15	—

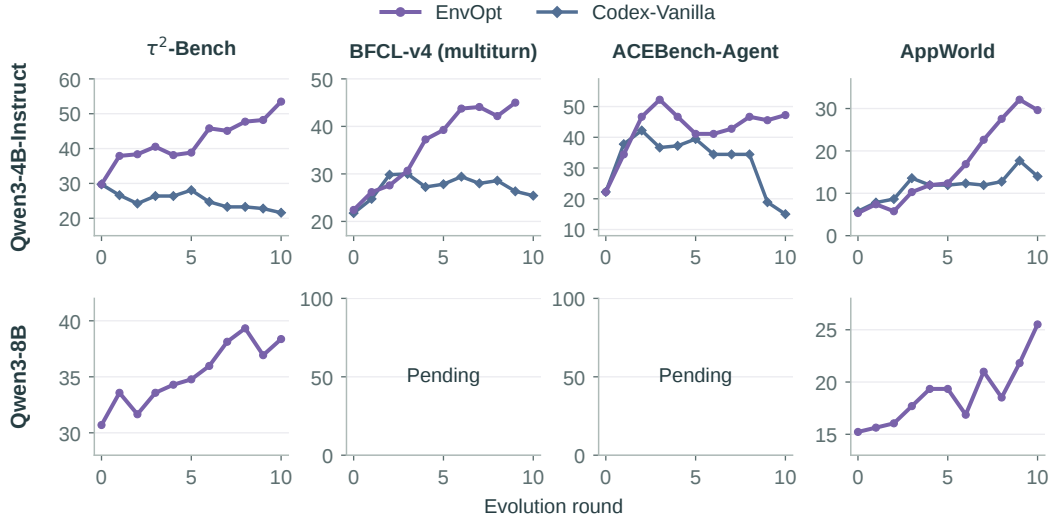


Figure 3: Public-evaluation performance over iterations 0–10 (R0: baseline), comparing ENVOPT with Codex-Vanilla; unobserved iterations are left blank. The 4B Codex ACEBench curve uses native tool calling. On τ^2 and ACEBench, both 4B curves use the ENVOPT R0 display reference. Qwen3-8B BFCL and ACEBench results are pending. AppWorld reports TGC.

These development-set results support more comprehensive, compact behavioral analysis in this setting. They do not isolate the hierarchy from the added coverage checks, match backbones or computation, or establish downstream RL gains. The diagnostic is separate from the historical 4B training runs.

5.4 TRAINING SIGNAL ACROSS EVOLUTION

We audit the saved RL reward groups from the original 4B experiments and Codex-Vanilla. A task provides within-group reward signal when its eight valid rollouts receive nonconstant rewards; we report the fraction of such tasks among complete, valid groups in each round (Figure 4b). This measures the relative reward contrast used by GRPO, retaining partial-credit rewards and excluding incomplete groups rather than treating them as failures.

Training success can exhaust the learning signal. On τ^2 , every Codex-Vanilla task in R9 and R10 receives eight rewards of one, leaving no within-group reward contrast. On BFCL, only 2 of 52 tasks retain reward variation in each of these rounds. In comparison, ENVOPT retains signal on 70.5% of valid τ^2 tasks and 35.5% of BFCL tasks in its final recorded round, R9. These observations

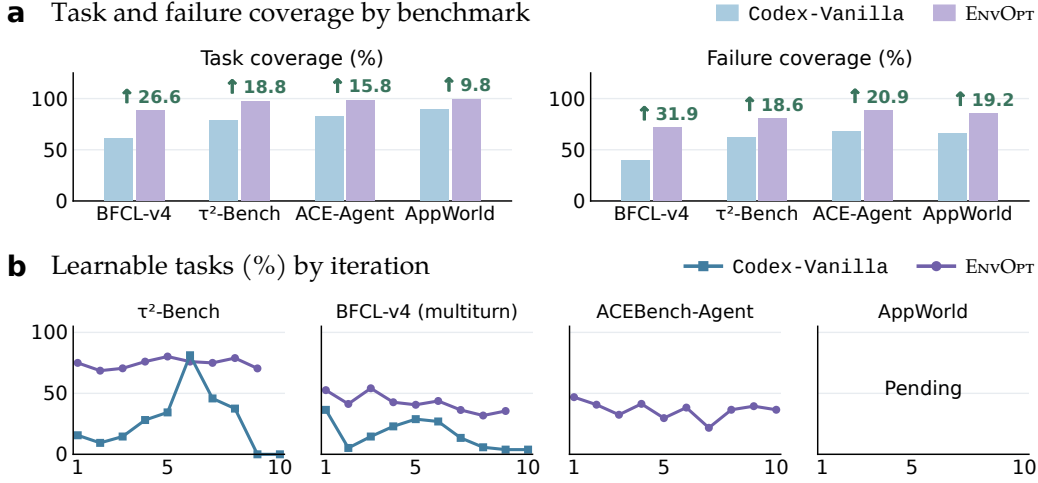


Figure 4: Research-process diagnostics. (a) Task and failure coverage of the coding-agent reports and refined map; $\uparrow$ labels show gains over `Codex-Vanilla` in percentage points. (b) Learnable tasks (nonconstant rewards across eight valid rollouts) by iteration; AppWorld remains pending in this panel.

motivate linking generated tasks to their actual training outcomes so the researcher can detect reward saturation and refresh its targets. They do not isolate the causal effect of context refinement.

5.5 ENVIRONMENT IMPLEMENTATION QUALITY

[TODO: Audit 20 sampled environments per benchmark, with eight rollouts each, using an independent LLM judge to flag implementation, interaction, and verifier defects. Report measured environment-level audit pass rates (no defects flagged in the inspected rollouts).]

5.6 REMAINING CONTROLS AND UNCERTAINTY

Controls still required. The available baseline records cover `Codex-Vanilla` and `Fixed-Env` results on all four 4B benchmarks. The offline diagnostic measures report quality, not a budget-matched isolation of the two designs. Such controls should match researcher, information, and training budgets. For behavioral understanding, compare selective case reading, a flat summary, and the source-traceable hierarchy, counting both analysis and researcher tokens. For context refinement, compare the same underlying records with and without target-linked organization, measuring evidence use and subsequent decisions under matched budgets and construction workflows. The proposed attention-guidance effect has not been isolated. An end-to-end native coding-agent control should retain equivalent tools and service access; the separately reported `Codex` curves do not establish such a matched control. Secondary controls compare a fixed library with fresh tasks, sampler/allocation-only adaptation, and program revision, or remove maintained experience. These causal controls remain unmeasured. SPADE-style or `EnvHarness`-style controls must additionally disclose differences in target-code and public-eval-feedback access.

Uncertainty and cost. Paired-task uncertainty is estimated using 10,000 bootstrap resamples of private-eval tasks within fixed subsets, keeping each task’s three repeats together and preserving the benchmark aggregation. These intervals measure finite-private-eval-set uncertainty conditional on a trained run, not training-seed variability. The recorded rounds contain 960, 809, and 626 distinct training Tasks for τ^2 , BFCL, and ACE respectively, with eight episodes per Task. Round counts do not fix training budgets: native ACE uses 96 Tasks in R1, 38 in R9, and 52 in R10. Final cost comparisons must also count analysis, planning, failed implementations, screening, serving, and evaluation, including queues and restarts. We make no end-to-end efficiency claim from task counts alone.

6 CONCLUSION

We study agents as data researchers that iteratively construct executable environments for model training. ENVOPT organizes this process around a traceable behavioral map and target-conditioned context refinement, supported by iterative environment construction. Our analyses identify gaps in behavioral understanding, feedback linkage, and environment implementation. Preliminary training results show useful checkpoints but also regressions, motivating controlled studies of sustained improvement.

AI USE STATEMENT

Generative AI assisted source-code and literature analysis, methodological formulation, drafting, and figure preparation. The research system uses language models for trajectory analysis, planning, program synthesis, solvability review and user simulation; generated-task rewards in the studied cohort are programmatic. Training results use existing experiment records. The offline diagnostic additionally generated and graded analysis reports from frozen public-evaluation trajectories; it did not rerun benchmark tasks or training. [TODO: The authors must verify this statement against the complete research history and review all AI-assisted claims and artifacts before submission.]

REPRODUCIBILITY STATEMENT

The research loop is specified in Section 3, construction interfaces and admission rules in Section 4, and evaluation access in Section 5. [TODO: Add the frozen implementation revision, resolved run configurations, complete artifacts, and an anonymous code supplement after the experiments are finalized.]

REFERENCES

- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik R Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=OC2z7iSQKa>.
- Chen Chen, Xinlong Hao, Weiwen Liu, Xu Huang, Xingshan Zeng, Shuai Yu, Dexun Li, Yuefeng Huang, Xiangcheng Liu, Wang Xinzhi, and Wu Liu. ACEBench: A comprehensive evaluation of LLM tool usage. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 12970–12998, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.697. URL <https://aclanthology.org/2025.findings-emnlp.697/>.
- Michael Dennis, Natasha Jaques, Eugene Vinitzky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 13049–13061. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/985e9a46e10005356bbaf194249f6856-Paper.pdf.
- Guanting Dong, Junting Lu, Junjie Huang, Wanjun Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, Jiajie Jin, Yutao Zhu, Hanbin Wang, Fangyu Lei, Qinyu Luo, Mingyang Chen, Zehui Chen, Jiazhan Feng, Ji-Rong Wen, and Zhicheng Dou. Agent-world: Scaling real-world environment synthesis for evolving general agent intelligence, 2026. URL <https://arxiv.org/abs/2604.18292>.
- Yaxin Du, Xiyuan Yang, Zhifan Zhou, Wanxu Liu, Zixing Lei, Zimeng Chen, Fenyi Liu, Haotian Wu, Yuzhu Cai, Zexi Liu, Xinyu Zhu, WenHao Wang, Linfeng Zhang, Chen Qian, and Siheng Chen. Datamaster: Data-centric autonomous ai research, 2026. URL <https://arxiv.org/abs/2605.10906>.

- Ruishan Fang, Siyuan Lu, Chenyi Zhuang, and Tao Lin. Rods: Reward-driven online data synthesis for multi-turn tool-use agents, 2026. URL <https://arxiv.org/abs/2606.19047>.
- Runnan Fang, Shihao Cai, Baixuan Li, Jialong Wu, Guangyu Li, Wenbiao Yin, Xinyu Wang, Xiaobin Wang, Liangcai Su, Zhen Zhang, Shibin Wu, Zhengwei Tao, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Towards general agentic intelligence via environment scaling, 2025. URL <https://arxiv.org/abs/2509.13311>.
- Hengrui Gu, Xiaotian Han, and Kaixiong Zhou. Writ: Write-read intensive trajectory synthesis for multi-turn user-facing agents, 2026. URL <https://arxiv.org/abs/2606.02908>.
- Chengsong Huang, Zifeng Wang, Rujun Han, Jun Yan, Yanfei Chen, Zoey CuiZhu, Ke Jiang, Peng Xia, Han Yu, Yufan Zhuang, Yifei Ming, Jiaqi Pan, Bhavana Dalvi Mishra, Jiaxin Huang, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. Envharness: Awakening static worlds for agent learning, 2026. URL <https://arxiv.org/abs/2608.19880>.
- Hao Jiang, Gangtao Xin, Yingdi Huang, Guojie Zhu, Jiangshan Zhang, Xinyuan Lin, Yunkun Xu, Chengyu Shen, Wenlong Fei, Jiawei Li, Yujie Fu, Sichen Kang, Tingyu Xie, Yedi Hu, Jingren Zhang, Hongcheng Gao, Jianshu Zeng, Chong Chen, Chang Guo, Chao Feng, Feng Wang, Fulin Lin, Jinchao Ma, Lang Mei, Li Huang, Liyan Liu, Qing He, Shuting Tao, Siyu Mo, Xiangnan Chen, Xiaohan Yu, Xiaoyang Li, Yanheng Hou, Yanyu Wu, Zhihan Yang, Wentao Zhang, Yang Gao, and Zhao Cao. Agentomnia: Scaling agentic models for full-scenario applications, 2026. URL <https://arxiv.org/abs/2607.23124>.
- Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 4940–4950. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/jiang21b.html>.
- Ilya Kulikov, Chenxi Whitehouse, Tianhao Wu, Yixin Nie, Swarnadeep Saha, Eryk Helenowski, Weizhe Yuan, Olga Golovneva, Jack Lanchantin, Yoram Bachrach, Jakob Foerster, Xian Li, Han Fang, Sainbayar Sukhbaatar, and Jason Weston. Autodata: An agentic data scientist to create high quality synthetic data, 2026. URL <https://arxiv.org/abs/2606.25996>.
- Bo Liu, Simon Yu, Yiding Jiang, Ao Qu, Andrew Zhao, Zichen Liu, Junsu Kim, Zijian Zhou, Seung-gone Kim, Tongzheng Ren, Mickel Liu, Hanfei Yu, Zhaorun Chen, Weiyan Shi, Paul Pu Liang, Luke Zettlemoyer, Yejin Choi, and Natasha Jaques. Spade: Self-play in adaptive synthetic executable environments, 2026. URL <https://arxiv.org/abs/2608.19197>.
- Fanqing Meng, Lingxiao Du, Qiguang Chen, Ziqi Zhao, Haocheng Lu, Mengkang Hu, and Michael Qizhe Shieh. Rsibench-data: Benchmarking data-centric research for recursive self-improvement, 2026. URL <https://arxiv.org/abs/2607.25886>.
- Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Kou Shi, Zun Wang, Qisheng Su, Shiting Huang, Ziao Zhang, Zhen Fang, Qingnan Ren, Jin Liu, Yu Zeng, Yiming Zhao, Lin Chen, Zehui Chen, and Feng Zhao. Facet: Preserving source intent and executable state in terminal task synthesis, 2026. URL <https://arxiv.org/abs/2608.18580>.

- Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. EnvScaler: Scaling tool-interactive environments for LLM agent via programmatic synthesis. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 8326–8357, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.407. URL <https://aclanthology.org/2026.findings-acl.407/>.
- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16022–16076, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.850. URL <https://aclanthology.org/2024.acl-long.850/>.
- Dunwei Tu, Hongyan Hao, Hansi Yang, Yihao Chen, Yu Yang, Yueqing Sun, Xingchen Liu, Furao Shen, Qi GU, Hui Su, and Xunliang Cai. Scaleenv: Scaling environment synthesis from scratch for generalist interactive tool-use agent training. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=4GX4r1CYtD>.
- Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=OvLnVkV14P>.
- Minrui Xu, Zilin Wang, Mengyi DENG, Zhiwei Li, Zhicheng Yang, Xiao Zhu, Yinhong Liu, Boyu Zhu, Baiyu Huang, Chao Chen, Heyuan Deng, Fei Mi, Lifeng Shang, Xingshan Zeng, and Zhi-jiang Guo. Envfactory: Scaling tool-use agents via executable environments synthesis and robust rl, 2026. URL <https://arxiv.org/abs/2605.18703>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Xingshan Zeng, Zishan Xu, Boju Zhang, Yuzhou Wu, Lingzhi Wang, Jianghao Lin, Liangyou Li, Yasheng Wang, Lifeng Shang, Xin Jiang, Weinan Zhang, Yong Yu, Qun Liu, and Weiwen Liu. What makes good agentic data? an ace lens on data generation for llm agents, 2026. URL <https://arxiv.org/abs/2608.27260>.

A CASE STUDIES OF CODEX-VANILLA LIMITATIONS

This section makes the limitations in the main text concrete through saved public-evaluation trajectories, synthetic training rollouts, reward groups, and generated programs from the four completed Qwen3-4B Codex-Vanilla studies. The examples below are lightly abridged for readability; synthetic credentials and opaque identifiers are omitted. They are diagnostic examples rather than causal proofs: each study has one training seed, and a later benchmark failure cannot be attributed to one generated environment without a controlled intervention.

Protocol scope. The ACEBench-Agent cases and scores in this section use the historical textual-interface Codex run, not the later native-function-calling run. These archived public-evaluation and training diagnostics do not supply new private/Test measurements.

A.1 CASE STUDY I: IMPROVEMENT WAS DISCOVERED BUT NOT RETAINED

Table 2: Public-evaluation trajectories. Every run discovers a checkpoint better than its final checkpoint. “Drop” is Final minus Peak.

Benchmark	Base	Peak	Final (R10)	Drop
τ^2 -Bench	27.10	28.06 (R5)	21.58	−6.47
BFCL-v4	21.75	30.00 (R3)	25.42	−4.58
ACEBench-Agent	10.56	22.22 (R7)	11.67	−10.56
AppWorld	5.76	17.70 (R9)	13.99	−3.70

Table 3: Representative task-level regressions. Each comparison uses the same public-evaluation task and three repeats at both checkpoints.

Benchmark task	and	Earlier successful behavior	Later failed behavior
τ^2			
Swap refund destinations across two orders		The agent tries the requested cross-order refund. The tool rejects it because refunds must use an eligible method. The agent makes no order mutation and transfers the case to a human. Result: 3/3 passes.	After the same rejection, the agent retries both returns using their original payment methods. The tools accept the returns, but the user’s requested refund destinations are not satisfied and both orders are now changed. Result: 0/3 passes.
BFCL			
Trade at an observed stock price, then cancel		The agent reads the current price as 310.23, places the order at 310.23, verifies it, and cancels it. Result: 3/3 passes.	The agent requests the price but places the order at 350.50 instead of the returned value, then cancels the wrong-price order. Cancellation does not repair the incorrect earlier action. Result: 0/3 passes.
ACEBench-Agent			
Order one pizza and record spending		The agent logs in, places one 88-yuan order, creates one reminder, and terminates. Result: 3/3 passes.	The agent places the requested 88-yuan order, repeats the same order, then places two more orders containing two pizzas each for 156 yuan. After spending 488 of the 500-yuan balance, it repeatedly retries the now-impossible order and full reminder until the turn limit. Result: 0/3 passes.
AppWorld			
Refund one accidental 56-dollar payment		The agent identifies the approved request, sends exactly 56 dollars back once, and completes the task. Result: 3/3 passes.	A representative repeat tries negative and zero amounts, then successfully sends 56 dollars three separate times while pursuing unnecessary receipt and contact operations. The task terminates normally but the state contains 168 dollars of outgoing refunds. Result: 0/3 passes.

These are not merely noisy score changes. The later policies lose different behavioral invariants: avoid an irreversible fallback that violates the user’s constraint; bind an action to a newly observed

value; stop after the requested side effect succeeds; and prevent duplicate financial actions. A data researcher that discovers a useful checkpoint but does not preserve these capabilities has not achieved sustained recursive improvement.

A.2 CASE STUDY II: AGGREGATE ANALYSIS HID THE FAILURE MECHANISM

The ACE example exposes what is lost when a trajectory is compressed to “failed by turn limit.” The first tool batch already completed the requested pizza order and reminder. The subsequent interaction is the failure:

Table 4: Abridged ACEBench-Agent trace from one failed final-checkpoint repeat.

Turn	Agent action	Environment result
1	Log in; inspect menu; order one Super Supreme Pizza; add spending reminder.	Order succeeds for 88 yuan; reminder succeeds. The requested external effects are already present.
2	Repeat login, menu lookup, the same pizza order, and the same reminder.	A second 88-yuan order succeeds; a duplicate reminder is added.
3	Order one Super Supreme and one Margherita pizza.	A third order succeeds for 156 yuan; 168 yuan remains.
4	Repeat the two-pizza order and reminder.	A fourth order succeeds for 156 yuan; 12 yuan remains and another reminder is added.
5–20	Repeat essentially the same two-pizza order and reminder batch.	The order fails for insufficient balance and the reminder fails because capacity is full; the same errors recur until the turn budget is exhausted.

The actionable mechanism is not generic “multi-step difficulty.” It is failure to recognize an already-satisfied goal, followed by duplicate irreversible actions and perseverance after stable errors. The other benchmarks show similarly specific mechanisms: the τ^2 failure confuses a policy-mandated refusal with permission to perform a different mutation; the BFCL failure violates an observation-to-action dependency; and the AppWorld failure repeats a successful transfer while exploring irrelevant recovery paths. Aggregate labels such as retail, Base, turn limit, or completed do not preserve these distinctions.

The researcher’s own available evidence demonstrates the consequence. In late τ^2 rounds, its retained summary contained domain rates and reward strata but no detailed RL failure trajectories. In late BFCL and ACE rounds, selected trajectories were excluded after integrity mismatches, leaving aggregate counts to drive allocation. AppWorld provides a before/after diagnostic: 144 permit-workflow episodes initially appeared only as constant low reward, so the researcher could not locate the failure. Once readable traces became available, one sampled task showed all eight attempts passing a human-readable roster alias as a canonical person identifier; this caused nine not-found errors, and the agent never attempted permit issuance. The next generated version added explicit alias-versus-ID guidance. This repair illustrates why the source behavior, rather than only its reward, must remain inspectable.

A.3 CASE STUDY III: TRAINING REWARD DID NOT TRACK LEARNING VALUE

Table 5: Reward-group composition over all ten rounds. A mixed group has nonzero reward variance across its eight valid rollouts and therefore supplies within-group contrast to GRPO. Constant partial groups have nonzero raw reward but zero normalized advantage.

Benchmark	Groups	Mixed	Mixed (%)	All 1	All 0	Const. partial
τ^2 -Bench	656	153	23.3	434	69	0
BFCL-v4	696	119	17.1	525	50	2
ACEBench-Agent	960	203	21.1	536	182	39
AppWorld	493	117	23.7	156	47	173

These cases separate three quantities that the loop must not conflate: raw reward, relative learning signal, and transfer to the target benchmark. High reward can be saturated; nonzero reward can be

Table 6: Late-round contradictions between synthetic training outcomes and public evaluation.

Benchmark	Synthetic training observation	What the benchmark showed
τ^2	The final two rounds contain 80 task groups and 640 rollouts; every reward is 1.0, so every normalized advantage is zero.	Public success declines from 23.26% to 22.78% and then 21.58%. Perfect synthetic completion neither provides a learning gradient nor certifies retained benchmark capability.
BFCL	Only 2 of 52 groups are mixed in each of the final two rounds. Synthetic strict success is 99.04% and 96.63%.	Public success declines from 28.58% to 26.33% and then 25.42%. The researcher continues reallocating among nearly saturated synthetic families while benchmark behavior worsens.
ACEBench-Agent	A closeout-aware round labels every Step episode as a strict failure unless the final assistant message contains a completion marker. Post-hoc state inspection finds 136 episodes with completed business outcomes but no marker.	The reward conflates doing the task with emitting protocol-specific text. It under-reports useful business behavior and changes subsequent allocation as if those objectives had zero success.
AppWorld	Four permit-workflow groups receive exactly 0.2 in all 32 rollouts. The 0.2 is initial-state attestation credit; one inspected task contains eight attempts with no tool call.	Nonzero mean reward looks like partial progress but supplies no within-group gradient and can occur without any action. Twelve of 28 final-round groups are constant partial.

preloaded; and low strict reward can be caused by a verifier convention rather than missing business competence. Without task-to-rollout-to-evaluation linkage, the researcher cannot reliably decide whether to retain, revise, or retire an environment.

A.4 CASE STUDY IV: EXECUTABLE ENVIRONMENTS IMPLEMENTED THE WRONG OR EASIER MECHANISM

ACEBench-Agent: a textual marker became part of task success. The intended target was to reduce looping after successful tool use. The generated Step environments instead assigned 80% of reward to the business state and the final 20% to a separate assistant response containing *finish conversation*. A completed tool workflow without that phrase received 0.8 and strict failure. Across one round, 14 supply, 49 cabinet, and 73 survey episodes completed their business outcomes without the marker, while no Step episode produced a successful closeout. The next revision removed this requirement for new tasks and restored outcome-only success. This is a concrete reward-definition mismatch: the environment was executable and internally consistent, but it trained a protocol phrase jointly with the intended behavior.

BFCL: private information leaked before the agent earned it. A parcel environment was supposed to reveal a private pickup window only through a user follow-up after an assistant message. A direct audit called the user callback with an empty history and received the equivalent of “Use pickup on October 7 in the morning.” The same audit found successful quote and dispatch operations recorded with rollback metadata even though state changes persisted. Both issues were repaired. The example shows why running the program and checking a successful endpoint is insufficient: interaction timing and trace semantics are part of the training mechanism.

AppWorld: identifiers encoded the answer. In the initial specimen generator, semantic roles were tied to fixed ID suffixes. Across tasks, ready target specimens occupied suffixes such as 7 and 18, the held distractor used 40, the archived distractor used 29, and the out-of-campaign specimen used 51. Compatible protocols used suffixes 0 and 1 while the incompatible protocol used 2; open destinations similarly used 0 and 1 while the closed destination used 2. Row order was shuffled, but a policy could still solve selection by suffix rather than inspecting status, campaign, compatibility, temperature, and destination fields. Independent review caught the shortcut, and the next genera-

tor randomized opaque IDs and role placement. The original tasks were valid programs, yet they instantiated a shortcut instead of the intended cross-application reasoning problem.

τ^2 : **the advertised constraints were slack.** The final synthetic family asked the policy to reserve a two-item laboratory bundle under compatibility, arrival-time, and total-price constraints, then re-plan after an inventory race invalidated the first quote. The generator’s own audit noted that arrival and price never became binding; distractors were distinguished mainly by capability or power. All 32 groups then scored 1.0, while all six earlier environment families were omitted from the round and public evaluation fell to 21.58%. This does not prove that the new family caused the decline. It does show that a plausible task description can collapse to an easier latent decision and that perfect reward on that task does not establish coverage of the claimed mechanism.

Takeaway. Together, the cases explain the paper’s limitations in operational terms. The researcher can miss the mechanism in a failure trace, receive reward that is uninformative or misdefined, validate a program that contains shortcuts or interaction defects, and then overwrite a useful checkpoint with a weaker one. Traceable behavioral evidence, explicit exposure and reward accounting, and independent implementation review are designed to make these failures visible; their individual causal effects still require matched ablations.